

6-DAY INSTRUCTOR-LED TECHNICAL TRAINING

Core Python with AI-Integrated Development

A fast-track, hands-on bootcamp for experienced developers: master Python's object model, data structures, and standard library — while working AI coding assistants into every stage of the development lifecycle.

- Prior dev experience required
- Python 3.14 · VS Code
- AI Coding Assistant hands-on labs

```
day_01.py

developer =
Engineer(role="builder")

assistant =
configure_ai_assistant()

for day in course.days:
    day.learn()
    day.build_with(assistant)
    day.ship()
```

6-Day Agenda at a Glance

D1

Day 1

Conceptual Foundations, AI Coding Assistant Setup & Python/String Fundamentals

D2

Day 2

AI Coding Agents, Control Flow, Iterators & Core Data Structures (Lists, Tuples, Sets)

D3

Day 3

Dictionaries, Nested & Extended Data Structures, and File Handling

D4

Day 4

Modules & Packages, OOP, Comprehensions, and Exceptions/Debugging/AI-Assisted Testing

D5

Day 5

Core Libraries, Automation, and AI Tools for Developers (Copilot, Context Engineering & Adoption)

D6

Day 6

Capstone Project + Evaluation

DAY 2

AI Coding Agents, Control Flow, Iterators & Core Data Structures (Lists, Tuples, Sets)

- AI Coding Agents for Development
- Iterator Protocol & For Loops
- Lists and Tuples II: List Manipulation
- Sets and Their Operations
- Branching & Conditional Logic
- Lists and Tuples I: Tuples & Unpacking
- Re-classification of Data Structures

What You'll Be Able To Do

1

Use AI coding agents (auto-completion, debugging, refactoring, querying) as part of a disciplined review workflow.

2

Write conditional logic using Python's comparison, logical, and membership operators, including elif chains and ternary expressions.

3

Explain and apply the iterator protocol (iter/next/StopIteration) and use every common for-loop form (list, dict, enumerate, zip, range, file, generator).

4

Manipulate tuples and lists correctly, including unpacking, extended unpacking, and in-place vs out-of-place operations.

5

Classify Python data structures as sequences vs non-sequences and explain how hashing powers dict/set lookups.

6

Apply set operations (union, intersection, difference, symmetric difference, subset/superset) to solve deduplication and relationship-query problems.

AI Coding Agents for Development

AI coding agents are IDE-integrated assistants (e.g. GitHub Copilot, Claude Code, Cursor) that generate, explain, and modify code from context and natural-language prompts.

THE PROBLEM IT SOLVES

Reduces time spent on boilerplate, repetitive debugging, and comprehension of unfamiliar code.

WHY IT MATTERS

Developers who use AI agents effectively ship features faster but must still validate correctness and security themselves.

WHERE IT'S USED

Used daily in IDEs (VS Code, JetBrains), CLI agents, and CI pipelines for review and test generation.

REAL-WORLD ANALOGY

"Like a very fast pair-programmer who knows every library's docs but has never run your code."

Core AI Agent Capabilities

AUTO-COMPLETION

- Context-aware suggestions from surrounding code, imports, and comments
- Multi-line and whole-function completions, not just single tokens
- Ranked by relevance to the current file plus open-tab context

CODE DEBUGGING

- Agent reads the traceback and surrounding code to localize the fault
- Suggests a root cause, not just a surface patch
- Can propose a fix and a regression test in one pass

CODE REFACTORING

- Simplifies or optimizes code while preserving observable behavior
- Extracts functions, renames for clarity, removes duplication
- Requires explicit instruction on what 'better' means (readability vs speed)

CODE QUERYING

- Ask the agent to explain a function, module, or diff in plain English
- Useful for onboarding onto unfamiliar or legacy codebases
- Answers are only as good as the context window the agent can see

AI Coding Agent Workflow

WHAT IS IT

A coding agent turns a natural-language prompt plus surrounding code context into a diff you review, test, and merge -- it never commits on its own.

WHY DEVELOPERS NEED IT

Structuring the interaction as a loop (prompt -> generate -> review -> test) keeps a human accountable for what ships.



AI Coding Agent Workflow — In Practice

EXAMPLE PROMPT

Refactor this function to remove duplicated validation logic, keep the same return type, and add type hints. Do not change its public signature.

before.py

```
def process(order):  
    if order['qty'] is None:  
        raise ValueError('qty required')  
    if order['qty'] <= 0:  
        raise ValueError('qty required')  
    if order['price'] is None:  
        raise ValueError('price required')  
    if order['price'] <= 0:  
        raise ValueError('price required')  
    return order['qty'] * order['price']
```

after_ai.py

```
def process(order: dict) -> float:  
    for field in ('qty', 'price'):  
        value = order.get(field)  
        if value is None or value <= 0:  
            raise ValueError(f'{field}'  
required')  
    return order['qty'] * order['price']
```

AI Coding Agent Workflow — Review & Practice

BEFORE YOU TRUST THE OUTPUT — REVIEW FOR:

Correctness

Security

Performance

Maintainability

Dependencies

Edge cases

HANDS-ON ACTIVITY

Paste a 15-line function with duplicated validation logic into your agent and ask it to refactor for readability without changing behavior.

Practical Example

```
practical_example.py

# Before -- duplicated conditional checks
def get_active_emails(users):
    result = []
    for u in users:
        if u['status'] == 'active':
            result.append(u['email'])
    return result

# After -- AI-suggested refactor (list comprehension)
def get_active_emails(users: list[dict]) -> list[str]:
    return [u['email'] for u in users if u['status'] == 'active']

users = [
    {"email": "alice@example.com", "status": "active"},
    {"email": "bob@example.com", "status": "suspended"},
    {"email": "carol@example.com", "status": "active"},
```

WHAT'S HAPPENING

- Same behavior and return type, no signature change
- List comprehension replaces the manual accumulator loop
- Type hints added for clarity and IDE tooling
- Agent flagged the original as verbose, not incorrect -- readability was the goal
- Still must be tested against empty lists and missing 'status' keys

CONSOLE OUTPUT

```
['alice@example.com', 'carol@example.com']
```

Best Practices

PEP 8 & READABILITY

- Ask the agent to match existing style/naming conventions
- Request docstrings and type hints on generated functions
- Keep prompts scoped to one function or module at a time

MAINTAINABILITY

- Never accept a refactor without reading the diff line-by-line
- Ask for the reasoning behind non-obvious changes

PERFORMANCE

- Confirm complexity didn't regress (e.g. $O(n)$ becoming $O(n^2)$)
- Benchmark before/after for hot-path code, not just correctness

SECURITY

- Never let an agent auto-insert credentials, tokens, or string-built SQL
- Review any new dependency the agent adds before installing it

Hands-on Exercise

PROBLEM STATEMENT

Take a 10-15 line function with nested if/else and a manual accumulator loop. Prompt an AI agent to refactor it for readability, then verify the output is behaviorally identical.

INPUT

A function that filters and transforms a list of dictionaries using nested conditionals.

EXPECTED OUTPUT

A refactored function (e.g. using a comprehension or guard clauses) that produces identical results on the same test inputs.

HINTS (no full solution here — give it a try first!)

- Write 2-3 test cases before refactoring so you can confirm behavior didn't change
- Ask the agent to explain each change it made
- Reject any change that alters the function's public signature unless you asked for that

Branching & Conditional Logic

Python expresses branching with if/elif/else blocks driven by boolean expressions built from comparison, logical, and membership operators.

THE PROBLEM IT SOLVES

Programs need to execute different code paths depending on runtime data.

WHY IT MATTERS

Correct use of and/or/not/in and Python's truthiness rules avoids subtle logic bugs.

WHERE IT'S USED

Input validation, request routing, business-rule engines, data filtering.

REAL-WORLD ANALOGY

"Same fork-in-the-road logic as any language, but Python's 'in' and truthiness rules are distinctive."

Operators for Branching

COMPARISON OPERATORS

- `==`, `!=`, `<`, `>`, `<=`, `>=` compare values, not identity
- `is` / `is not` compare identity (same object in memory)
- Chained comparisons: `0 < x < 10` is valid Python

LOGICAL OPERATORS

- `and`, `or`, `not` operate on truthiness, not just booleans
- `and`/or short-circuit and return an operand, not always `True`/`False`
- `not` negates the truthiness of any object

MEMBERSHIP OPERATORS

- `in` / `not in` test membership in list, tuple, set, dict, or str
- Works on any iterable; `'in'` on a dict checks keys, not values
- Often replaces a manual loop written to check existence

DEVELOPER PERSPECTIVE

- `and`/or return one of their operands (e.g. `x or default`) -- a common Python idiom
- Falsy values include `0`, `''`, `[]`, `{}`, `None` -- not just `False`

if / elif / else and Ternary

```
syntax.py

if condition_1:
    do_a()
elif condition_2:
    do_b()
else:
    do_c()

# Ternary (conditional expression)
result = value_if_true if condition else value_if_false
```

HOW IT READS

- No parentheses or braces required -- indentation defines the block
- elif chains replace switch/case (Python had no match statement before 3.10)
- Ternary is a single expression, useful for simple value selection
- A colon plus an indented block is mandatory after each clause
- Nested ternaries are legal but hurt readability -- avoid stacking them

Basic Example

```
example.py

age = 20
has_id = True

if age >= 21:
    status = "full access"
elif age >= 18 and has_id:
    status = "limited access"
else:
    status = "denied"

print(status)
```

WHAT'S HAPPENING

- elif checks the second condition only if the first is False
- and requires both operands to be truthy
- Ternary alternative: status = 'ok' if age >= 18 else 'denied'

CONSOLE OUTPUT

```
limited access
```

Practical Example

```
practical_example.py

def validate_config(cfg: dict) -> str:
    if "host" not in cfg or "port" not in cfg:
        return "missing required keys"
    if not (0 < cfg["port"] < 65536):
        return "invalid port"
    env = cfg.get("env", "dev")
    label = "production" if env == "prod" else "non-production"
    return f"config OK ({label})"

print(validate_config({"host": "db1", "port": 5432, "env": "prod"}))
```

WHAT'S HAPPENING

- 'not in' checks both required keys are present before touching them
- Chained comparison `0 < port < 65536` validates a range in one expression
- `.get()` with a default avoids a `KeyError` for the optional 'env' key
- Ternary picks a label without a full if/else block

CONSOLE OUTPUT

```
config OK (production)
```

Common Mistakes

✗ INCORRECT

```
if x == True:  
    ...
```

✗ INCORRECT

```
if not x == None:  
    ...
```

✗ INCORRECT

```
if a or b == 5:  
    ...
```

✗ INCORRECT

```
if key in dict.keys():  
    ...
```

✗ INCORRECT

```
if lst == []:  
    ...
```

✓ CORRECT

```
if x:  
    ...
```

✓ CORRECT

```
if x is not None:  
    ...
```

✓ CORRECT

```
if a == 5 or b == 5:  
    ...
```

✓ CORRECT

```
if key in dict:  
    ...
```

✓ CORRECT

```
if not lst:  
    ...
```

Common Mistakes — Why It Matters

- Mistake 1: Comparing explicitly to True/False is redundant; test truthiness directly.
- Mistake 2: Identity checks against None must use is / is not, not equality.
- Mistake 3: 'or' doesn't distribute across comparisons -- each side needs its own comparison.
- Mistake 4: 'in' already checks keys on a dict; .keys() adds unnecessary overhead.
- Mistake 5: Empty containers are falsy -- a truthiness check is idiomatic and works for any container type.

Best Practices

PEP 8 & READABILITY

- Prefer truthiness checks (if items:) over explicit comparisons
- Keep ternaries to one simple condition; use if/elif for anything more

MAINTAINABILITY

- Extract complex boolean conditions into a well-named variable or function
- Avoid deeply nested if/elif -- return early (guard clauses) instead

PERFORMANCE

- Short-circuiting means ordering matters: put cheap/likely-false checks first

AI-Assisted Development

PROMPT

This nested if/elif chain is hard to follow -- suggest a flatter version using guard clauses and Python's truthiness rules.

```
before_ai.py

def check(order):
    if order is not None:
        if order['qty'] > 0:
            if order['price'] > 0:
                return True
            else:
                return False
        else:
            return False
    else:
        return False
```

AI ASSISTANT SUGGESTS

- Flattened to guard clauses that return early on invalid input
- Replaced 'order is not None' with a truthiness check where safe
- Combined the three failure paths into one boolean expression
- Suggested a docstring stating what a valid order requires

DEVELOPER MUST STILL VERIFY

- Confirm the guard clauses don't change behavior on edge cases like qty == 0
- Check the flattened version still handles order being an empty dict
- Re-run existing unit tests before merging

Hands-on Exercise

PROBLEM STATEMENT

Write a function `classify(temp, is_raining)` that returns 'stay in', 'bring umbrella', or 'go out' using `elif` and the `and/or/in` operators.

INPUT

```
classify(15, True), classify(25, False), classify(5, False)
```

EXPECTED OUTPUT

```
'bring umbrella', 'go out', 'stay in' (or your own reasonable  
rule set)
```

HINTS (no full solution here — give it a try first!)

- Use chained comparisons for temperature ranges
- Combine `is_raining` with `and/or` rather than nesting ifs
- Consider a ternary for the simplest branch only

Iterator Protocol & For Loops

Every Python for-loop is sugar over the iterator protocol: `iter()` produces an iterator, then repeated `next()` calls pull values until `StopIteration`.

THE PROBLEM IT SOLVES

Explains how for-loops actually work under the hood, and why some objects are iterable while others aren't.

WHY IT MATTERS

Understanding the protocol lets you build custom iterables/generators and explains errors like 'object is not iterable'.

WHERE IT'S USED

Every for-loop, comprehension, and generator -- file reading, streaming API responses, lazy data pipelines.

REAL-WORLD ANALOGY

"An iterator is a bookmark in a book: `iter()` opens the book, `next()` turns one page at a time, `StopIteration` is reaching the back cover."

Iterable vs Iterator

KEY TERMINOLOGY

- Iterable -- an object with `__iter__()`, e.g. list, dict, str, range
- Iterator -- an object with `__next__()`, produced by `iter(iterable)`
- `StopIteration` -- exception an iterator raises once exhausted
- `next(it)` -- pulls the next value from an iterator manually

HOW IT WORKS

- `for x in obj` calls `iter(obj)` once, then `next()` on each pass
- The for-loop catches `StopIteration` for you -- you never see it directly
- An iterator is itself iterable (its `__iter__` returns itself)

IMPORTANT RULES

- Once exhausted, an iterator can't be reset -- you must create a new one
- Lists/dicts are iterable but are not themselves iterators

DEVELOPER PERSPECTIVE

- 'TypeError: object is not iterable' means no `__iter__` is defined
- Generators (`yield`) are the easiest way to write your own iterator

The Iterator Protocol



for-loop Forms

```
syntax.py

for item in some_list:
    ...

for key, value in some_dict.items():
    ...

for index, item in enumerate(some_list):
    ...

for a, b in zip(list_a, list_b):
    ...

for i in range(5):
    ...

with open("file.txt") as f:
    for line in f:
        ...

for value in (x * x for x in range(5)):
    ...
```

HOW IT READS

- `dict.items()` yields (key, value) tuples -- unpack directly in the loop
- `enumerate()` gives (index, value) pairs without a manual counter
- `zip()` pairs up multiple iterables, stopping at the shortest one
- An open file object is itself an iterator over its lines
- The last form iterates a generator expression lazily

Basic Example

example.py

```
names = ["Ann", "Bo", "Cy"]
for i, name in enumerate(names, start=1):
    print(i, name)
```

WHAT'S HAPPENING

- `enumerate(names, start=1)` numbers items from 1 instead of 0
- The (index, value) tuple is unpacked directly in the for-clause

CONSOLE OUTPUT

```
1 Ann
2 Bo
3 Cy
```

Practical Example

practical_example.py

```
timestamps = ["09:01", "09:03", "09:07"]
messages = ["start", "warn", "stop"]
config = {"retries": 3, "timeout": 30}

for ts, msg in zip(timestamps, messages):
    print(f"[{ts}] {msg}")

for key, value in config.items():
    print(f"{key} = {value}")
```

WHAT'S HAPPENING

- `zip()` pairs timestamps with messages positionally
- `dict.items()` loops keys and values together, no `config[key]` lookups needed
- Both loops rely on `__iter__` defined on list and dict respectively

CONSOLE OUTPUT

```
[09:01] start
[09:03] warn
[09:07] stop
retries = 3
timeout = 30
```

Advanced / Developer Example

```
advanced_example.py

def batch(iterable, size):
    it = iter(iterable)
    while True:
        chunk = []
        for _ in range(size):
            try:
                chunk.append(next(it))
            except StopIteration:
                if chunk:
                    yield chunk
                return
        yield chunk

for group in batch(range(7), 3):
    print(group)
```

WHAT'S HAPPENING

- `iter(iterable)` explicitly creates the iterator this function drives manually
- `next(it)` is wrapped in `try/except` that catches `StopIteration` directly
- `batch()` is a generator (contains `yield`) -- it IS an iterator when called
- The partial final chunk is yielded before the generator returns

CONSOLE OUTPUT

```
[0, 1, 2]
[3, 4, 5]
[6]
```

Common Mistakes

✗ INCORRECT

```
it = [1, 2, 3]
next(it)
```

✓ CORRECT

```
it = iter([1, 2, 3])
next(it)
```

✗ INCORRECT

```
for line in open('f.txt').readlines():
    ...
```

✓ CORRECT

```
for line in open('f.txt'):
    ...
```

✗ INCORRECT

```
for i in range(len(items)):
    print(items[i])
```

✓ CORRECT

```
for item in items:
    print(item)
```

✗ INCORRECT

```
it = iter(data)
for x in it:
    ...
for x in it:
    ...
```

✓ CORRECT

```
for x in data:
    ...
for x in data:
    ...
```

✗ INCORRECT

```
for k in my_dict:
    print(k, my_dict.values())
```

✓ CORRECT

```
for k, v in my_dict.items():
    print(k, v)
```

Common Mistakes — Why It Matters

- Mistake 1: A list is iterable but not itself an iterator -- `next()` needs an iterator from `iter()`.
- Mistake 2: Iterating the file object directly streams lines lazily instead of loading them all into memory.
- Mistake 3: Iterate the object directly unless you specifically need the index (use `enumerate` for that).
- Mistake 4: An exhausted iterator stays exhausted; re-iterate the original iterable, not the spent iterator.
- Mistake 5: Iterating a dict alone yields only keys; use `.items()` to get key/value pairs.

Best Practices

PEP 8 & READABILITY

- Prefer 'for item in collection' over index-based loops
- Use enumerate()/zip() instead of manual counters or indexing

MAINTAINABILITY

- Write generators (yield) for custom sequences instead of building full lists in memory
- Name loop variables meaningfully (row, user), not just x

PERFORMANCE

- Iterate files and large datasets lazily -- avoid .readlines() / list(huge_iterable)
- zip() and enumerate() are implemented in C and faster than manual indexing

AI-Assisted Development

PROMPT

This loop reads an entire file into memory before processing. Rewrite it to iterate lazily and stop early once a match is found.

```
before_ai.py

def find_error(path):
    lines = open(path).readlines()
    for i in range(len(lines)):
        if "ERROR" in lines[i]:
            return i, lines[i]
    return None
```

AI ASSISTANT SUGGESTS

- Suggested iterating the file object directly instead of `.readlines()`
- Replaced the index-based loop with `enumerate()` for line numbers
- Confirmed the existing early return already short-circuits correctly
- Recommended a context manager (with `open(...)`) to ensure the file closes

DEVELOPER MUST STILL VERIFY

- Confirm behavior is identical on an empty file
- Check the fix still closes the file handle (use `with`)
- Benchmark on a large log file to confirm the memory improvement

Hands-on Exercise

PROBLEM STATEMENT

Write a generator function `evens_up_to(n)` that yields even numbers from 0 up to `n`, then consume it manually with `iter()/next()` for the first two values before finishing with a for-loop.

INPUT

```
evens_up_to(10)
```

EXPECTED OUTPUT

```
First two next() calls print 0 and 2; the following for-loop  
prints 4, 6, 8, 10
```

HINTS (no full solution here — give it a try first!)

- A function containing `yield` is automatically a generator (an iterator)
- Only call `iter()` on it if you're unsure it's already an iterator
- Wrap manual `next()` calls in `try/except StopIteration` if you don't know the length

Lists and Tuples I: Tuples & Unpacking

Tuples are immutable, ordered sequences; unpacking assigns their elements to multiple variables in one statement.

THE PROBLEM IT SOLVES

Need a lightweight, immutable container for fixed-shape records and multiple return values.

WHY IT MATTERS

Tuples are hashable (usable as dict keys/set members) and signal 'this data won't change' to other developers.

WHERE IT'S USED

Function multi-returns, dict keys for composite lookups, coordinates, database rows.

REAL-WORLD ANALOGY

"A tuple is a sealed, labeled box -- you can look inside but can't swap out its contents."

Sequence Operations & Tuple Basics

COMMON SEQUENCE OPERATIONS

- Indexing: `seq[0]`, `seq[-1]` (negative indexes count from the end)
- Slicing: `seq[1:3]`, `seq[::2]`, `seq[::-1]` (reversed)
- Concatenation: `seq1 + seq2`; membership: `x in seq`; `len(seq)`
- Identical syntax works on lists, tuples, and strings

TUPLE CHARACTERISTICS

- Immutable: no item assignment, append, or sort after creation
- Defined with commas: `(1, 2, 3)` -- parentheses are optional but recommended
- A single-element tuple needs a trailing comma: `(1,)` not `(1)`

IMPORTANT RULES

- Tuples are hashable only if every element inside is hashable
- 'Immutable' means the tuple's references can't change -- a list inside a tuple can still mutate

DEVELOPER PERSPECTIVE

- Use a tuple to say 'this is a fixed record', a list to say 'this may grow or shrink'

Tuple Creation & Unpacking

```
syntax.py

point = (3, 4)
x, y = point

record = ("Ann", 29, "Berlin")
name, age, city = record

first, *rest = [1, 2, 3, 4]
*init, last = [1, 2, 3, 4]
a, *middle, b = [1, 2, 3, 4]

single = (5,)
empty = ()
```

HOW IT READS

- Parentheses around a tuple literal are optional; the commas make it a tuple
- Basic unpacking requires the variable count to match the element count
- `*rest` captures a variable number of remaining items as a list
- A starred name can appear anywhere in the unpacking target, only once
- A trailing comma is mandatory to create a one-element tuple

Basic Example

```
example.py

coords = (10, 20, 30)
print(coords[0])
print(coords[-1])
print(coords[0:2])
print(len(coords))
print(20 in coords)
```

WHAT'S HAPPENING

- Indexing and slicing work exactly like lists/strings
- Negative index -1 accesses the last element
- Slicing returns a new tuple; the original is untouched either way since tuples can't mutate

CONSOLE OUTPUT

```
10
30
(10, 20)
3
True
```

Practical Example

practical_example.py

```
def min_max(values):  
    return min(values), max(values)  
  
low, high = min_max([4, 9, 1, 7])  
print(low, high)  
  
row = ("2024-01-01", "INFO", "service started", "web-01")  
date, level, *details = row  
print(date, level, details)
```

WHAT'S HAPPENING

- `min_max` returns a tuple (`low`, `high`) -- Python packs multiple return values automatically
- The caller unpacks it directly into `low`, `high`
- `*details` collects every field after the first two into a list
- Common pattern for parsing fixed-prefix, variable-tail records

CONSOLE OUTPUT

```
1 9  
2024-01-01 INFO ['service started', 'web-01']
```

Advanced / Developer Example

```
advanced_example.py

def distance(p1: tuple, p2: tuple) -> float:
    (x1, y1), (x2, y2) = p1, p2
    return ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5

a, b = (0, 0), (3, 4)
print(distance(a, b))

a, b = b, a
print(a, b)
```

WHAT'S HAPPENING

- Nested unpacking: `(x1, y1), (x2, y2) = p1, p2` destructures two tuples at once
- A tuple like `(0, 0)` is a natural fixed 2-field record for a coordinate
- `a, b = b, a` swaps values via unpacking -- no temp variable needed
- The swap works because the right-hand side is fully evaluated into a tuple before assignment

CONSOLE OUTPUT

```
5.0
(3, 4) (0, 0)
```

Common Mistakes

✗ INCORRECT

```
t = (1)
type(t)
```

✓ CORRECT

```
t = (1,)
type(t)
```

✗ INCORRECT

```
point[0] = 99
```

✓ CORRECT

```
point = (99,) + point[1:]
```

✗ INCORRECT

```
a, b, c = (1, 2)
```

✓ CORRECT

```
a, b = (1, 2)
```

✗ INCORRECT

```
*rest = [1, 2, 3]
```

✓ CORRECT

```
*rest, = [1, 2, 3]
```

✗ INCORRECT

```
d = {}
d[[1, 2]] = 'x'
```

✓ CORRECT

```
d[(1, 2)] = 'x'
```

Common Mistakes — Why It Matters

- Mistake 1: Without the trailing comma, parentheses are just grouping -- not a tuple.
- Mistake 2: Tuples don't support item assignment; build a new tuple instead.
- Mistake 3: Unpacking requires the exact target count to match the tuple length (unless using *rest).
- Mistake 4: A single starred target needs a trailing comma to be valid unpacking syntax.
- Mistake 5: Dict keys must be hashable; use a tuple instead of a list as a composite key.

Best Practices

PEP 8 & READABILITY

- Use tuples for heterogeneous, fixed-shape data (name, age); lists for homogeneous, variable-length data
- Name unpacked variables descriptively, not a, b, c

MAINTAINABILITY

- Prefer a named alternative (namedtuple/dataclass) once a tuple's fields gain meaning beyond position

PERFORMANCE

- Tuples are slightly faster to create and iterate than lists due to immutability

AI-Assisted Development

PROMPT

This function returns a plain tuple of 4 values with no field names -- suggest how to make the return value self-documenting without breaking existing unpacking call sites.

before_ai.py

```
def stats(values):  
    return min(values), max(values),  
    sum(values), len(values)
```

AI ASSISTANT SUGGESTS

- Flagged that callers must remember the positional meaning of each field
- Suggested collections.namedtuple or typing.NamedTuple as a drop-in, still-unpackable replacement
- Noted existing 'lo, hi, total, n = stats(x)' call sites keep working unchanged
- Recommended documenting field order in a docstring regardless of the choice

DEVELOPER MUST STILL VERIFY

- Confirm the namedtuple import doesn't clash with an existing name elsewhere
- Re-run callers to confirm positional unpacking still works
- Decide if the extra readability is worth the added import

Hands-on Exercise

PROBLEM STATEMENT

Write a function `parse_line(line)` that takes `'name,age,city,country,zip'` and returns `(name, age, rest)` using extended unpacking to separate the first two fields from the remainder.

INPUT

```
parse_line('Ann,29,Berlin,Germany,10115')
```

EXPECTED OUTPUT

```
name='Ann', age='29', rest=['Berlin', 'Germany', '10115']
```

HINTS (no full solution here — give it a try first!)

- `str.split(',')` gives you a list to unpack
- Use `*rest` to capture everything after the first two fields
- Return a tuple so the caller can unpack it too

Lists and Tuples II: List Manipulation

Lists are mutable, ordered sequences supporting in-place growth, shrinkage, and reordering.

THE PROBLEM IT SOLVES

Most real data (rows fetched from an API, accumulating results) needs to grow, shrink, or reorder after creation -- tuples can't do that.

WHY IT MATTERS

Mutability is powerful but introduces aliasing bugs; understanding in-place vs out-of-place operations avoids them.

WHERE IT'S USED

Building result sets, queues, buffers -- any collection that changes over its lifetime.

REAL-WORLD ANALOGY

"A list is a whiteboard you erase and rewrite freely; a tuple is a printed page."

Mutability & List Operations

MUTABILITY & ALIASING

- Two variables can reference the SAME list object (aliasing)
- Mutating through one name is visible through the other
- `list(other)` or `other.copy()` creates an independent shallow copy

CORE MUTATING METHODS

- `append(x)` adds one item at the end; `extend(iterable)` adds many
- `insert(i, x)` adds at a position; `pop(i)` removes and returns an item
- `remove(x)` deletes the first matching value by value, not index

SEARCH / SORT / REVERSE

- `list.sort()` sorts in place and returns `None`; `sorted(list)` returns a new list
- Both accept `key=` (a function) and `reverse=True`
- `list.reverse()` reverses in place; `reversed(list)` returns an iterator

DEVELOPER PERSPECTIVE

- Sets are also mutable but unordered/unindexed -- covered as their own topic later
- If a function shouldn't mutate its input, copy it first or use `sorted()/reversed()`

List Methods

```
syntax.py

items = [3, 1, 2]
items.append(4)
items.extend([5, 6])
items.insert(0, 0)
removed = items.pop()
items.remove(1)

items.sort()
newly_sorted = sorted(items, reverse=True)
items.reverse()
backwards = list(reversed(items))
```

HOW IT READS

- `append/extend/insert/pop/remove` all mutate the list in place (`pop` also returns a value)
- `sort()` mutates and returns `None` -- never do `x = items.sort()`
- `sorted()` leaves the original untouched and returns a new list
- `key=` (not shown here) takes a function applied to each element before comparing
- `reversed()` returns a lazy iterator -- wrap it in `list()` to see all values

Basic Example

example.py

```
cart = ["pen"]
cart.append("book")
cart.extend(["eraser", "ruler"])
cart.insert(0, "bag")
removed = cart.pop()
print(cart)
print(removed)
```

WHAT'S HAPPENING

- `append` adds a single item; `extend` adds each item of an iterable
- `insert(0, ...)` places 'bag' at the front
- `pop()` with no index removes and returns the LAST item

CONSOLE OUTPUT

```
['bag', 'pen', 'book', 'eraser']
ruler
```

Practical Example

```
practical_example.py

records = [
    {"name": "Bo", "score": 72},
    {"name": "Ann", "score": 91},
    {"name": "Cy", "score": 85},
]

top_first = sorted(records, key=lambda r: r["score"], reverse=True)
print([r["name"] for r in top_first])

records.sort(key=lambda r: r["name"])
print([r["name"] for r in records])
```

WHAT'S HAPPENING

- `sorted(..., key=...)` builds a new list ordered by score, descending
- The original 'records' list is untouched by `sorted()`
- `records.sort(key=...)` then reorders the SAME list object, this time by name
- `key=` receives one element and returns the value to compare on

CONSOLE OUTPUT

```
['Ann', 'Cy', 'Bo']
['Ann', 'Bo', 'Cy']
```

Advanced / Developer Example

advanced_example.py

```
def add_item(cart, item):
    cart.append(item)  # mutates the caller's list!
    return cart

original = ["pen", "book"]
alias = original
new_ref = add_item(original, "coupon")

print(original is new_ref)
print(alias)

safe_copy = original.copy()
safe_copy.append("local-only")
print(original)
print(safe_copy)
```

WHAT'S HAPPENING

- `add_item` mutates the list it's given -- the caller's list changes too
- `'alias'` sees the mutation because it references the SAME list object
- `original is new_ref` is `True` -- `append()` never created a new list
- `.copy()` makes an independent shallow copy so later mutation stays local
- This is exactly why in-place vs out-of-place matters for shared/aliased data

CONSOLE OUTPUT

```
True
['pen', 'book', 'coupon']
['pen', 'book', 'coupon']
['pen', 'book', 'coupon', 'local-only']
```

In-Place vs Out-of-Place Operations

Operation	In-Place (mutates)	Out-of-Place (new object)	Notes
Sort	<code>list.sort()</code>	<code>sorted(list)</code>	<code>sort()</code> returns None; never assign it
Reverse	<code>list.reverse()</code>	<code>reversed(list)</code> / <code>list[::-1]</code>	<code>reversed()</code> returns an iterator, not a list
Add items	<code>list.append(x)</code> / <code>.extend(y)</code>	<code>list + other_list</code>	<code>'+'</code> allocates a brand-new list
Remove item	<code>list.pop(i)</code> / <code>.remove(x)</code>	<code>[x for x in lst if x != target]</code>	Filtering builds a fresh list, original stays intact

Common Mistakes

✗ INCORRECT

```
items = items.sort()
```

✓ CORRECT

```
items.sort()
```

✗ INCORRECT

```
b = a
b.append(1)  # a changes too
```

✓ CORRECT

```
b = a.copy()
b.append(1)  # a unaffected
```

✗ INCORRECT

```
for x in items:
    if x < 0:
        items.remove(x)
```

✓ CORRECT

```
items = [x for x in items if x >= 0]
```

✗ INCORRECT

```
def f(lst):
    lst = lst + [1]
    return lst
```

✓ CORRECT

```
def f(lst):
    return lst + [1]
```

✗ INCORRECT

```
items.insert(0, x)  # inside a large loop
```

✓ CORRECT

```
items.append(x)  # then reverse once, if order matters
```

Common Mistakes — Why It Matters

- Mistake 1: `sort()` returns `None`; assigning it overwrites your list with `None`.
- Mistake 2: Plain assignment aliases the same list object; use `copy()` when you need independence.
- Mistake 3: Mutating a list while iterating over it skips elements; build a filtered copy instead.
- Mistake 4: Reassigning the parameter is redundant here -- return the new list directly instead of an extra rebinding step.
- Mistake 5: `insert(0, x)` is $O(n)$ per call; repeated front-inserts on a big list are much slower than appending.

Best Practices

PEP 8 & READABILITY

- Use list comprehensions instead of manual append loops where it stays readable
- Prefer `sorted()`/`reversed()` when you must not touch the caller's data

MAINTAINABILITY

- Document whether a function mutates its list argument in its docstring
- Return a new list from pure functions instead of mutating inputs

PERFORMANCE

- `append()` is $O(1)$ amortized; `insert(0, x)` is $O(n)$ -- avoid it in hot loops
- In-place `sort()` avoids allocating a second list when you don't need the original

AI-Assisted Development

PROMPT

This function mutates the list its caller passed in, which caused a bug elsewhere. Fix it so the caller's original list is never touched.

before_ai.py

```
def top_n(scores, n):  
    scores.sort(reverse=True)  
    return scores[:n]
```

AI ASSISTANT SUGGESTS

- Identified that `scores.sort()` mutates the caller's original list in place
- Replaced it with `sorted(scores, reverse=True)` to avoid touching the input
- Suggested a docstring noting the function is now pure (no side effects)
- Flagged that any caller relying on the old mutation would need updating

DEVELOPER MUST STILL VERIFY

- Search the codebase for callers that depended on the old in-place sort side effect
- Confirm performance is acceptable now that a new list is allocated
- Add a unit test asserting the input list is unchanged after the call

Hands-on Exercise

PROBLEM STATEMENT

Given a list of order dicts with 'amount' and 'status', write a function that returns a NEW list of orders with status 'paid', sorted by amount descending, without mutating or reordering the original list.

INPUT

```
orders = [{'amount': 50, 'status': 'paid'}, {'amount': 120, 'status': 'pending'}, {'amount': 80, 'status': 'paid'}]
```

EXPECTED OUTPUT

```
[{'amount': 80, 'status': 'paid'}, {'amount': 50, 'status': 'paid'}] (original 'orders' list order unchanged)
```

HINTS (no full solution here — give it a try first!)

- Filter with a list comprehension, then sort the result -- don't call `.sort()` on the original
- Use `key=lambda o: o['amount']` with `reverse=True`
- Verify with `orders == original_copy` after your function runs

Re-classification of Data Structures

Python's built-in containers split into ordered Sequences (list, tuple, str, range) and unordered Non-Sequences (set, dict), the latter built on hashing.

THE PROBLEM IT SOLVES

Choosing the right category affects which operations are valid (indexing/slicing) and how fast membership tests run.

WHY IT MATTERS

Understanding hashing explains why dict/set lookups are $O(1)$ average and why only hashable objects can be keys or set members.

WHERE IT'S USED

Every dict/set usage, caching layers, database indexing, deduplication pipelines.

REAL-WORLD ANALOGY

"A sequence is a numbered coat-check rack; a hash-based structure is a library card catalog -- no numbered slots, just a computed lookup."

Sequences vs Non-Sequences

SEQUENCES

- Ordered, indexable by integer position: list, tuple, str, range
- Support slicing, concatenation, and repetition (seq * n)
- Preserve insertion order and allow duplicates

NON-SEQUENCES

- set, frozenset, dict -- no positional index, no slicing
- Backed by a hash table for near $O(1)$ average lookup/insert
- set members and dict keys must be unique and hashable

IMPORTANT RULES

- seq[i] only works on Sequences -- sets/dicts raise TypeError on indexing
- Dict preserves insertion order for iteration (3.7+) but is still non-sequence: no dict[0]

DEVELOPER PERSPECTIVE

- Pick a Sequence when order/position matters; a Non-Sequence when uniqueness/lookup speed matters

How Hashing Works



Case Study: Hashing

KEY TERMINOLOGY

- Hash key -- the object being hashed (must be immutable/hashable)
- Hashing function -- `hash(x)` maps an object to an integer
- Hash table -- an array indexed by `hash(key)` reduced to a table position

HOW IT WORKS

- `hash(key)` computes an integer fingerprint in $O(1)$
- The integer is reduced to a table index (based on table size)
- The value is stored in, or looked up from, the bucket at that index

PROBLEMS: COLLISIONS

- Two different keys can hash to the same index -- a collision
- Python resolves collisions internally (open addressing) -- transparent to you
- A poor hash distribution -> many collisions -> lookup degrades toward $O(n)$

DEVELOPER PERSPECTIVE

- Equal objects MUST have equal hashes: `a == b` implies `hash(a) == hash(b)`
- Never define `__eq__` on a class without also defining a consistent `__hash__`

Basic Example

example.py

```
print(hash(42))  
print(hash((1, 2, 3)))  
print(hash(42) == hash(42.0))
```

WHAT'S HAPPENING

- `hash()` returns the integer fingerprint dict/set internals use
- This tuple hashes deterministically here because every element (an int) is hashable
- Equal values hash equally even across types: `42 == 42.0`

CONSOLE OUTPUT

```
42  
529344067295497451  
True
```

Practical Example

```
practical_example.py

cache = {}
cache[("user", 42)] = "Ann"
print(cache[("user", 42)])

try:
    cache[["user", 42]] = "Bo"
except TypeError as e:
    print("Error:", e)
```

WHAT'S HAPPENING

- A tuple ('user', 42) is hashable, so it's a valid composite dict key
- A list is mutable, so it's unhashable -- using one as a key raises TypeError
- This is exactly why fixed-record data (see the Tuples topic) should be a tuple, not a list, when used as a key

CONSOLE OUTPUT

```
Ann
Error: unhashable type: 'list'
```

Common Mistakes

✗ INCORRECT

```
d = {}  
d[[1, 2]] = 'x'
```

✓ CORRECT

```
d[(1, 2)] = 'x'
```

✗ INCORRECT

```
class Point:  
    def __eq__(self, other):  
        ...  
    # no __hash__ defined
```

✓ CORRECT

```
class Point:  
    def __eq__(self, other):  
        ...  
    def __hash__(self):  
        ...
```

✗ INCORRECT

```
s = {1, 2}  
d = {s: 'x'}
```

✓ CORRECT

```
d = {frozenset(s): 'x'}
```

Common Mistakes — Why It Matters

- Mistake 1: Lists are mutable and therefore unhashable; use a tuple as a composite key instead.
- Mistake 2: Defining `__eq__` without `__hash__` makes instances unhashable in Python 3 -- sets/dicts reject them.
- Mistake 3: A set is itself mutable/unhashable; use frozenset when you need a hashable set-like key.

Best Practices

PEP 8 & READABILITY

- Choose set/dict when membership or key lookup matters more than order
- Choose list/tuple when position or duplicate values matter

MAINTAINABILITY

- Document why a type was chosen when it isn't the obvious default (e.g. a tuple key in a dict)

PERFORMANCE

- 'x in set_or_dict' is $O(1)$ average; 'x in list' is $O(n)$ -- this matters at scale

AI-Assisted Development

PROMPT

Explain why this code raises `TypeError: unhashable type: 'list'`, and suggest the minimal fix.

```
before_ai.py

seen = {}
for row in rows:
    key = [row['user_id'], row['day']]
    seen[key] = True
```

AI ASSISTANT SUGGESTS

- Identified that 'key' is a list, and lists can't be dict keys because they're mutable/unhashable
- Suggested changing key to a tuple: `(row['user_id'], row['day'])`
- Noted no other code changes are required since tuples support the same equality checks
- Flagged that this pattern is common in deduplication logic

DEVELOPER MUST STILL VERIFY

- Confirm no downstream code assumes 'key' supports list mutation (e.g. `key.append(...)`)
- Re-run tests that exercise the deduplication path
- Check whether the same bug pattern exists elsewhere in the codebase

Hands-on Exercise

PROBLEM STATEMENT

Given a list of (lat, lon) tuples representing GPS pings, build a dict mapping each unique coordinate tuple to how many times it appears. Then show what happens if you try the same using a list instead of a tuple as the key.

INPUT

```
pings = [(1.0, 2.0), (1.0, 2.0), (3.0, 4.0)]
```

EXPECTED OUTPUT

```
{(1.0, 2.0): 2, (3.0, 4.0): 1}, followed by a TypeError when  
a list is used as a key
```

HINTS (no full solution here — give it a try first!)

- Use `dict.get(key, 0) + 1` to count occurrences, or `collections.Counter`
- Deliberately try `counts[[lat, lon]] = ...` to trigger and observe the `TypeError`
- Relate the error back to `hash()/hashability` rules from this topic

Sets and Their Operations

A set is a mutable, unordered collection of unique, hashable elements supporting fast membership tests and mathematical set operations.

THE PROBLEM IT SOLVES

Deduplicating data and testing membership or relationships between collections efficiently.

WHY IT MATTERS

Sets give $O(1)$ average membership checks and built-in union/intersection/difference instead of manual loops.

WHERE IT'S USED

Deduplication, permission/role checks, tag systems, graph algorithms, finding common/differing records between datasets.

REAL-WORLD ANALOGY

"A set is a bag of unique tokens -- order doesn't matter, and you can only ever have one of each token."

Non-Sequence: Pros & Cons

PROS

- $O(1)$ average membership test (`x in my_set`) vs $O(n)$ for a list
- Automatic deduplication -- duplicates are silently discarded
- Direct support for mathematical set operations (union, intersection, ...)

CONS

- Unordered -- no indexing (`set[0]` is invalid), no guaranteed iteration order
- Elements must be hashable -- no lists/dicts/other sets as members
- More memory overhead per element than an equivalent list

IMPORTANT RULES

- `set()` creates an empty set -- `{}` creates an empty DICT, not a set
- `frozenset` is the immutable, hashable counterpart (usable as a dict key or set member)

DEVELOPER PERSPECTIVE

- Reach for a set the moment you catch yourself writing 'if x not in list: list.append(x)'

Creating & Using Sets

 syntax.py

```
s1 = {1, 2, 3}
s2 = set([2, 3, 4])
empty = set()  # NOT {}

s1.add(5)
s1.discard(1)   # no error if missing
s1.remove(2)    # raises KeyError if missing

frozen = frozenset({1, 2, 3})
```

HOW IT READS

- `{}` is an empty dict; `set()` is the only way to create an empty set
- `add()/discard()/remove()` mutate the set in place, like list methods
- `discard()` is safe on a missing element; `remove()` is not
- `frozenset` is hashable -- usable as a dict key or as a member of another set

Set Operators vs Methods

Operation	Operator	Method Equivalent	Notes
Union	<code>a b</code>	<code>a.union(b)</code>	All elements from both sets
Intersection	<code>a & b</code>	<code>a.intersection(b)</code>	Elements present in both sets
Difference	<code>a - b</code>	<code>a.difference(b)</code>	In a but not in b
Symmetric Difference	<code>a ^ b</code>	<code>a.symmetric_difference(b)</code>	In a or b, but not both

Basic Example

```
example.py
```

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

WHAT'S HAPPENING

- `a | b` (or `a.union(b)`) combines all unique elements from both sets
- `a & b` (or `a.intersection(b)`) keeps only elements common to both
- `a - b` keeps elements in `a` that are absent from `b`; `a ^ b` keeps the non-overlapping elements

CONSOLE OUTPUT

```
{1, 2, 3, 4}
{2, 3}
{1}
{1, 4}
```

Practical Example

```
practical_example.py

user_a_perms = {"read", "write", "delete"}
user_b_perms = {"read", "write"}

print(user_b_perms.issubset(user_a_perms))
print(user_a_perms.issuperset(user_b_perms))
print(sorted(user_a_perms.intersection(user_b_perms)))
print(sorted(user_a_perms.difference(user_b_perms)))
```

WHAT'S HAPPENING

- `issubset()` checks if every element of `user_b_perms` is also in `user_a_perms`
- `issuperset()` is the mirror check -- does `user_a_perms` contain all of `user_b_perms`
- `intersection()`/`difference()` are used here for a permissions comparison, a common relationship query
- `sorted()` is applied only to make the printed output order deterministic for this slide -- sets themselves are unordered

CONSOLE OUTPUT

```
True
True
['read', 'write']
['delete']
```

Advanced / Developer Example

```
advanced_example.py

log_ids = ["a1", "b2", "a1", "c3", "b2", "a1"]
unique_ids = set(log_ids)
print(len(log_ids), len(unique_ids))

allowed_roles = {"admin", "editor"}
user_roles = {"editor", "viewer"}
if user_roles & allowed_roles:
    print("access granted")
else:
    print("access denied")

friends_a = {"Bo", "Cy", "Dee"}
friends_b = {"Cy", "Dee", "Eli"}
mutual = sorted(friends_a & friends_b)
print(mutual)
```

WHAT'S HAPPENING

- `set(log_ids)` deduplicates in one call -- 6 raw entries collapse to 3 unique IDs
- `user_roles & allowed_roles` is truthy if there's any overlap -- a fast relationship/permission check
- `friends_a & friends_b` finds mutual connections between two users
- These three lines cover the classic set use cases: dedup, membership testing, relationship queries

CONSOLE OUTPUT

```
6 3
access granted
['Cy', 'Dee']
```

Common Mistakes

✗ INCORRECT

```
s = {}
```

✓ CORRECT

```
s = set()
```

✗ INCORRECT

```
my_set[0]
```

✓ CORRECT

```
next(iter(my_set))
```

✗ INCORRECT

```
s = {[1, 2], [3, 4]}
```

✓ CORRECT

```
s = {(1, 2), (3, 4)}
```

✗ INCORRECT

```
a.remove(x) # x might not exist
```

✓ CORRECT

```
a.discard(x) # safe if x is missing
```

✗ INCORRECT

```
for i, x in enumerate(my_set):  
    ...
```

✓ CORRECT

```
for x in sorted(my_set):  
    ...
```

Common Mistakes — Why It Matters

- Mistake 1: `{}` creates an empty dict, not an empty set.
- Mistake 2: Sets are unordered and unindexed -- there is no positional access.
- Mistake 3: Set elements must be hashable; use tuples instead of lists.
- Mistake 4: `remove()` raises `KeyError` on a missing element; `discard()` does not.
- Mistake 5: Enumerating a set implies an order/position that doesn't reliably exist; sort first if order matters.

Best Practices

PEP 8 & READABILITY

- Use set literals {a, b} when non-empty; reserve set() for empty sets
- Prefer 'x in a_set' over manual duplicate-checking loops

MAINTAINABILITY

- Convert back to a list (often sorted) at output boundaries so downstream order is stable

PERFORMANCE

- Use a set for membership tests on large collections -- $O(1)$ average vs $O(n)$ for a list
- Use frozenset for read-only lookup tables that must also be hashable

AI-Assisted Development

PROMPT

This function deduplicates a list using a manual loop with 'if item not in seen'. Suggest a faster approach for large inputs.

```
before_ai.py

def dedupe(items):
    seen = []
    result = []
    for item in items:
        if item not in seen:
            seen.append(item)
            result.append(item)
    return result
```

AI ASSISTANT SUGGESTS

- Flagged 'item not in seen' as $O(n)$ per check on a list, $O(n^2)$ overall
- Suggested tracking 'seen' as a set for $O(1)$ average membership checks
- Noted order-preserving dedup still works: check/add to the set, append to the result list
- Offered `list(dict.fromkeys(items))` as a one-line alternative if order-preserving dedup is the only goal

DEVELOPER MUST STILL VERIFY

- Confirm all items are hashable before switching to a set-based approach
- Benchmark on a realistic input size to confirm the improvement
- Check whether the original insertion order needs to be preserved in the output

Hands-on Exercise

PROBLEM STATEMENT

Given two lists of customer IDs from two different data sources, find: (1) IDs present in both, (2) IDs only in source A, (3) IDs in either but not both. Also deduplicate each source list first.

INPUT

```
source_a = ['c1', 'c2', 'c3', 'c2']  
source_b = ['c2', 'c3', 'c4']
```

EXPECTED OUTPUT

```
both={'c2', 'c3'}, only_a={'c1'}, symmetric_diff={'c1', 'c4'}
```

HINTS (no full solution here — give it a try first!)

- Convert each list to a set first -- this also deduplicates it
- Use `&`, `-`, and `^` (or their method equivalents) for the three results
- Sort results before printing for a stable, readable order

Day Summary

CONCEPTS LEARNED

- AI agent workflow: prompt -> generate -> review -> test -> production
- Truthiness and short-circuit evaluation in branching
- The iterator protocol underlying every for-loop
- Sequence vs non-sequence data structures
- Hashing and hash tables as the basis for dict/set performance
- Mutability, aliasing, and in-place vs out-of-place operations

KEY SYNTAX

- if/elif/else, ternary expressions
- and/or/not, in/not in, chained comparisons
- for ... in, enumerate(), zip(), range(), iterating files/generators
- Tuple/list indexing, slicing, unpacking, *rest extended unpacking
- list.append/pop/insert/extend/sort vs sorted()
- Set literals, |, &, -, ^ and their method equivalents

KEY TAKEAWAYS

- AI agents accelerate coding but never replace developer review of correctness, security, and performance
- Prefer iterating objects directly over manual indexing
- Immutability (tuples) communicates intent; mutability (lists) requires aliasing awareness
- Sets/dicts trade ordering for O(1) average lookup via hashing

PRACTICAL SKILLS GAINED

- Structuring effective AI coding-agent prompts for refactors and debugging
- Writing idiomatic Python branching and looping code
- Building and consuming custom iterators/generators
- Choosing the right container (list/tuple/set) for a given data shape
- Reasoning about hashability and hash collisions
- Performing set-based deduplication and relationship queries

Log Record Processor: Iterating, Fixed Records & Deduplication

Your team ingests raw server log lines as fixed-format strings and must produce a deduplicated, sorted summary report for the on-call dashboard.

REQUIREMENTS

- Parse each raw log line into a fixed-shape tuple (timestamp, level, service, message) using `str.split` and unpacking.
- Iterate all records with a for-loop using `enumerate()` to also report each record's line number.
- Use branching (`if/elif`) to classify each record's level as 'critical', 'warning', or 'info'.
- Collect all unique service names into a set to avoid duplicates.
- Use extended unpacking (`*rest`) to separate any log line that has extra trailing fields.
- Sort the final collection of unique services using `sorted()`, without mutating the original list of raw lines.
- Print a summary: total records, count per level, and the sorted list of unique services.

CONSTRAINTS

- Do not use any external libraries -- only built-ins (`str.split`, tuples, sets, `sorted`, `enumerate`)
- The original `raw_lines` list must be unchanged after processing
- Each parsed record must be represented as a tuple, not a list

Log Record Processor: Iterating, Fixed Records & Deduplication — Input / Output

SAMPLE INPUT

```
raw_lines = [  
    "09:01,ERROR,billing,payment failed",  
    "09:02,INFO,billing,invoice sent",  
    "09:03,WARNING,auth,slow response",  
    "09:04,ERROR,auth,token expired,retry=3",  
    "09:05,INFO,billing,invoice sent",  
]
```

EXPECTED OUTPUT

```
Line 1: [09:01] ERROR billing -> critical  
Line 2: [09:02] INFO billing -> info  
Line 3: [09:03] WARNING auth -> warning  
Line 4: [09:04] ERROR auth -> critical (extra fields:  
['retry=3'])  
Line 5: [09:05] INFO billing -> info
```

```
Total records: 5
```

```
critical: 2, warning: 1, info: 2
```

```
Unique services: ['auth', 'billing']
```

HINTS

- `str.split(',')` then tuple unpacking with `*rest` handles the variable-length trailing fields
- A dict or an `if/elif` chain works for counting per-level totals
- Add each service name to a set as you go, then `sorted()` it once at the end

OPTIONAL CHALLENGE

Extend the report to also compute, using set operations, which services appear in ERROR-level records but never in INFO-level records.